Toyl Programming Language

Sai Hemanth Bheemreddy
Georgia Institute of Technology

This is a living document for me to plan and implement my toy-ish programming language, named “toyl” for “TOY programming Language.” (pronounced same as "toil" cause it’s gonna be a pain in my a** to get done.)

1. Learn Compilers
2. Learn the math behind computer science
3. Eventually have a turing-complete programming language that supports Functional+Graph Operations
4. Eventually have the compiler be self-hosted
5. Learn technical writing

The plan is to first have a basic turing-complete language with a custom backend and later switch to using LLVM/MLIR backend when I start playing around with Functional Programming Concepts or Graph Operations.

1 Technical Stack

This section is a high-level overview of the compiler’s internals.

1.1 v0.1

The compiler is written in Rust using chumsky module for frontend lexer and parser. I choose Rust simply because I wanted to get better in the language. The reason for choosing a parser generator was to make it easier for me to change the language grammar easily since in the early stages, I expect the grammar to change frequently.

Add more details as implementation progresses.

2 High Level Design Decisions

Complete this section

- Scanning & Parsing
- Static Analysis
- IR Design

- Optimizations
- Code Gen
- Runtime Implementation
- JIT (At the very end)

3 Roadmap

3.1 May-Aug 2026

- Basic Language Specification for v0.1
- Lexer and Parser
- Symbol Table and Semantic Analysis
- Writeup Specification for Intermediate Representation
- Instruction Section - inspired from LLVM's GlobalISel
- Register Allocation
- Instruction Scheduling
- Complete Backend for macOS